Rapport Technique

Service de Gestion des Véhicules

Microservice Spring Boot — CRUD, Kafka, GraphQL, Tests et Observabilité

Système de Gestion de Flotte de Véhicules

Master 1 GIL — 2025/2026

Informations du projet

Auteurs :	Kherbachi Rima & Mouhoubi Leiticia
Formation :	Master 1 GIL
Université :	Université de Rouen Normandie
Encadrants :	Lydia et Luc
Année :	2025 – 2026
Semaines documentées :	1 et 2

Mars 2026

Chapitre 1

Introduction

Ce rapport décrit en détail l'implémentation du microservice **vehicule-service**, un composant central du système de gestion de flotte de véhicules. Ce service gère le cycle de vie complet des véhicules : création, consultation, mise à jour et suppression (CRUD), tout en intégrant des technologies modernes pour la communication asynchrone (Kafka), les API flexibles (GraphQL), l'observabilité (OpenTelemetry) et les tests automatisés.

1.1 Architecture générale

Le service suit une architecture en couches classique :

1. **Couche Contrôleur** — Expose les API REST et GraphQL
2. **Couche Service** — Contient la logique métier
3. **Couche Repository** — Gère la persistance avec Spring Data JPA
4. **Couche Événementielle** — Publie et consomme des événements via Kafka

Port du service : 4000
Base de données : PostgreSQL (fleet_vehicles)
Messagerie : Apache Kafka (localhost :9092)
Tracing : OpenTelemetry (OTLP sur localhost :4317)

1.2 Représentations graphiques de l'architecture

Pour mieux visualiser la structure logicielle et les flux de communication au sein du microservice, les deux diagrammes suivants détaillent l'implémentation de notre solution.

1.2.1 Flux de communication et Composants

Le diagramme ci-dessous illustre le flux des requêtes depuis les clients (via les API REST ou GraphQL) jusqu'à la base de données PostgreSQL et le bus de messages Kafka. Il met en évidence le rôle central de la couche Service (`VehicleServiceImpl`) qui orchestre la logique métier, interagit avec le `Repository` pour la persistance, et délègue l'envoi d'événements asynchrones au `Producer` Kafka.

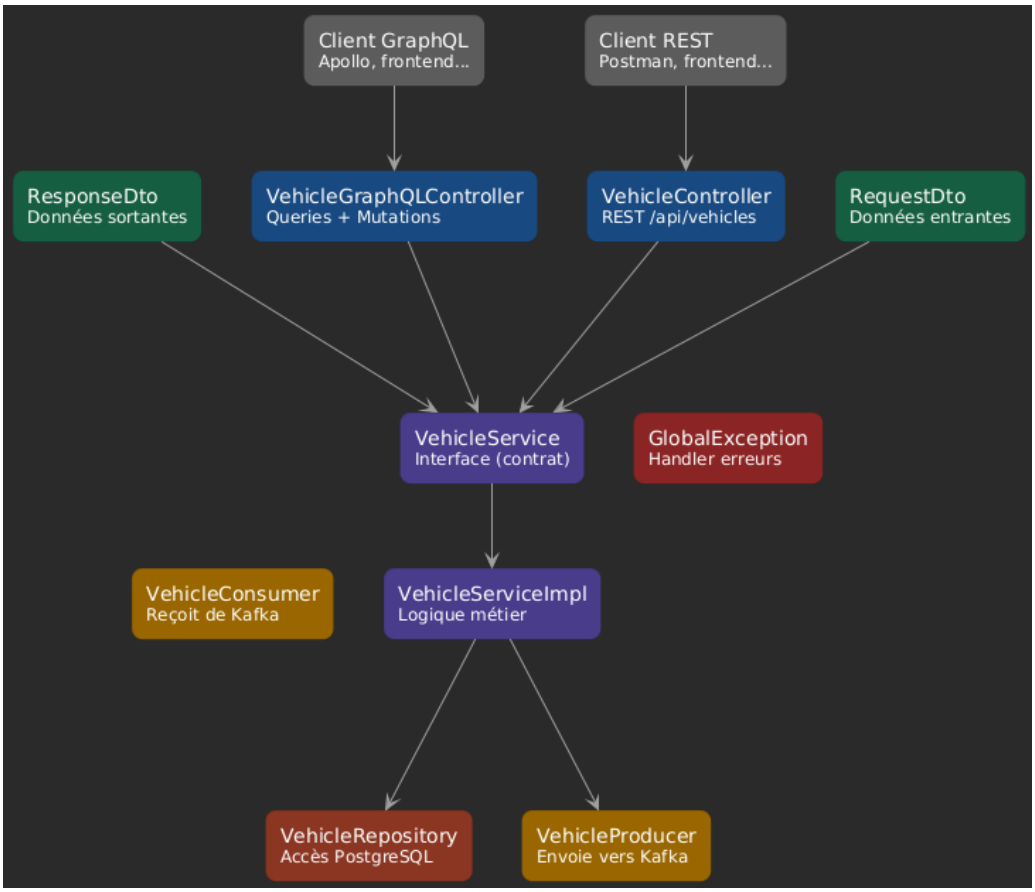


FIGURE 1.1 – Architecture détaillée et flux de données du microservice Véhicules

1.2.2 Diagramme de classes complet

La figure suivante présente l'organisation exhaustive des classes et interfaces du microservice, réparties par packages (`controllers`, `services`, `repository`, `entity`, `dto`, `kafka`, `exception`). Ce diagramme souligne l'utilisation des DTOs pour isoler l'entité JPA `Vehicle` des interfaces externes, garantissant ainsi un couplage faible et une meilleure maintenabilité.

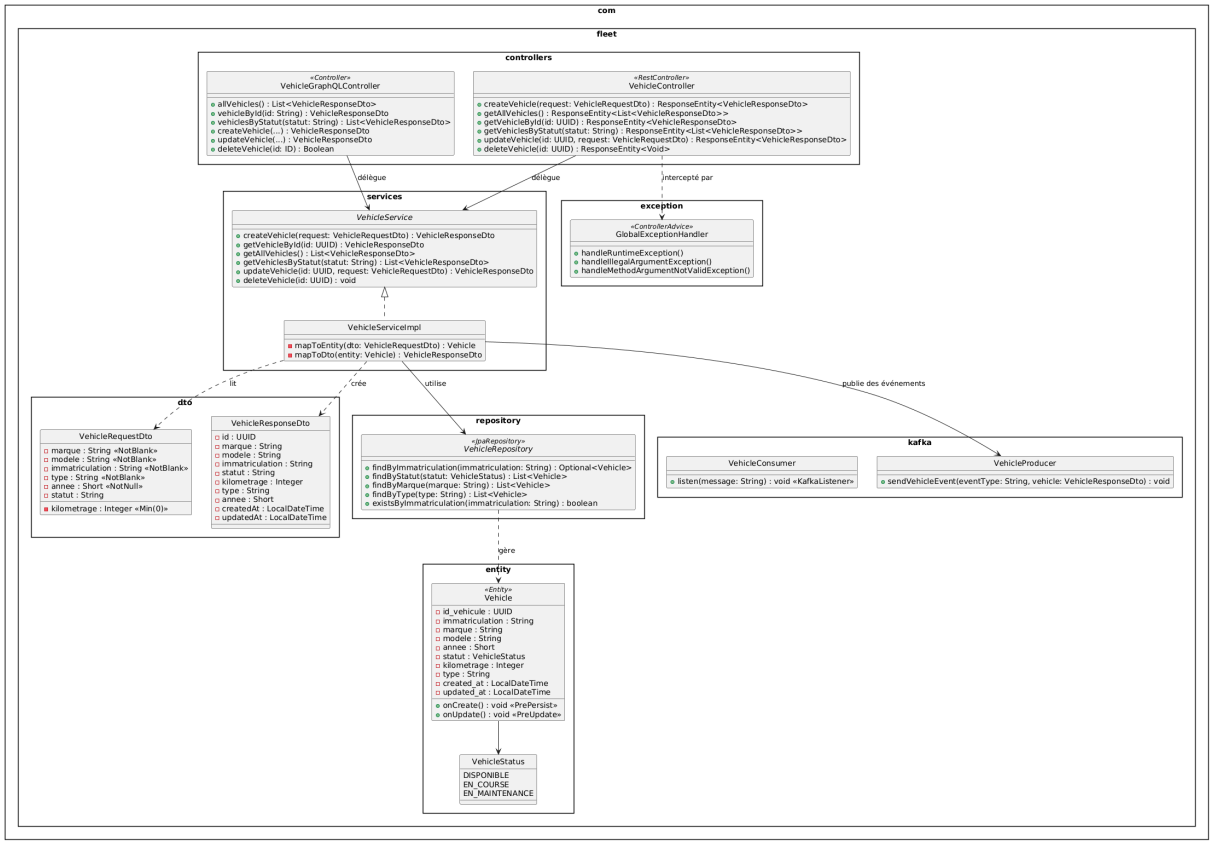


FIGURE 1.2 – Diagramme de classes complet du microservice Véhicules

Chapitre 2

Interface REST et Alignement OpenAPI

Le service expose une interface REST alignée sur la spécification OpenAPI de la passerelle (`gateway/openapi/vehicule-openapi.yaml`). Les points d'entrée utilisent le nommage en français (`/api/vehicules`) et le champ identifiant est `id_vehicule`.

2.1 Points d'entrée (Endpoints)

Endpoints REST — Base path : /api/vehicules

Méthode	Endpoint	Description
POST	/	Création d'un véhicule
GET	/	Liste de tous les véhicules
GET	/disponibles	Liste uniquement les véhicules disponibles
GET	/{id}	Détail d'un véhicule par UUID
PUT	/{id}	Mise à jour complète
PATCH	/{id}/statut	Mise à jour du statut (Kafka)
PATCH	/{id}/kilometrage	Mise à jour du kilométrage
DELETE	/{id}	Suppression du véhicule

2.2 Structure des données (DTO)

Les réponses utilisent la structure suivante pour garantir la compatibilité avec le schéma SQL et la passerelle :

```
{
  "id_vehicule": "550e8400-e29b-41d4-a716-446655440000",
  "immatriculation": "AB-123-CD",
  "marque": "Renault",
  "modele": "Clio",
  "annee": 2022,
  "statut": "DISPONIBLE",
  "kilometrage": 45000,
  "type": "Berline"
}
```

Listing 2.1 – Exemple de réponse VehicleResponseDto

2.3 Ports et Connectivité

Le microservice est configuré pour s'intégrer dans un environnement distribué :

— **Port Interne (Service)** : `4000` (configuré dans `application.yml`)

— **Port Externe (Gateway)** : `3001` (port exposé par la passerelle pour les clients externes)

Chapitre 3

Bibliothèques et dépendances

Le projet utilise **Spring Boot 3.1.5** comme framework principal.

3.1 Dépendances principales

Dépendances de production

Bibliothèque	Artifact	Rôle
Spring Boot Web	`spring-boot-starter-web`	Serveur HTTP embarqué (Tomcat), API REST
Spring Data JPA	`spring-boot-starter-data-jpa`	ORM Hibernate, mapping entité-table
PostgreSQL Driver	`postgresql`	Pilote JDBC pour PostgreSQL
Validation	`spring-boot-starter-validation`	Validation des DTOs (`@NotBlank`, `@Min`)
Spring Kafka	`spring-kafka`	Producteur/consommateur Kafka
Spring GraphQL	`spring-boot-starter-graphql`	API GraphQL avec schéma SDL
Actuator	`spring-boot-starter-actuator`	Endpoints de santé et métriques
Micrometer	`micrometer-registry-prometheus`	Export des métriques vers Prometheus
OpenTelemetry	`opentelemetry-spring-boot-starter`	Tracing distribué

3.2 Dépendances de test

Dépendances de test

Bibliothèque	Artifact	Rôle
Spring Boot Test	`spring-boot-starter-test`	JUnit 5, Mockito, MockMvc
H2 Database	`h2`	Base de données en mémoire pour les tests
Spring Kafka Test	`spring-kafka-test`	Kafka embarqué pour les tests
Spring GraphQL Test	`spring-graphql-test`	Outils de test GraphQL

Chapitre 4

Modèle de données

4.1 Schéma SQL (PostgreSQL)

La table `vehicule` est définie avec les contraintes suivantes :

```sql
CREATE TYPE statut_vehicule AS ENUM (
    'DISPONIBLE', 'EN_COURSE', 'EN_MAINTENANCE'
);

CREATE TABLE vehicule (
    id_vehicule     UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    immatriculation VARCHAR(20)      NOT NULL UNIQUE,
    marque          VARCHAR(100)     NOT NULL,
    modele          VARCHAR(100)     NOT NULL,
    annee           SMALLINT         NOT NULL
                    CHECK (annee >= 1900
                    AND annee <= EXTRACT(YEAR FROM NOW()) + 1),
    statut          statut_vehicule  NOT NULL DEFAULT 'DISPONIBLE',
    kilometrage     INTEGER          NOT NULL DEFAULT 0
                    CHECK (kilometrage >= 0),
    type            VARCHAR(50)      NOT NULL,
    created_at      TIMESTAMPTZ      NOT NULL DEFAULT NOW(),
    updated_at      TIMESTAMPTZ
);
```

Listing 4.1 – Script SQL de création de la table vehicule

Points clés du schéma :

- **UUID** comme clé primaire, généré via `gen_random_uuid()`
- **Enum PostgreSQL** pour le statut, garantissant l'intégrité
- **Contraintes CHECK** sur l'année et le kilométrage
- **Unicité** de l'immatriculation
- **Index** sur statut et immatriculation

4.2 Entité JPA

L'entité Java `Vehicle` est mappée sur la table SQL :

```java
@Entity
@Table(name = "vehicule")
public class Vehicle {

    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    @Column(name = "id_vehicule")
    private UUID id;

    @Column(name = "immatriculation", nullable = false,
            unique = true, length = 20)
    private String immatriculation;

    @Column(nullable = false, length = 100)
    private String marque;

    @Column(nullable = false, length = 100)
    private String modele;

    @Column(name = "annee", nullable = false)
    private Short annee;

    @Column(nullable = false)
    @Enumerated(EnumType.STRING)
    private VehicleStatus statut = VehicleStatus.DISPONIBLE;

    @Column(nullable = false)
    private Integer kilometrage = 0;

    public enum VehicleStatus {
        DISPONIBLE, EN_COURSE, EN_MAINTENANCE
    }

    @PrePersist
    protected void onCreate() {
        this.createdAt = LocalDateTime.now();
        this.updatedAt = LocalDateTime.now();
    }

    @PreUpdate
    protected void onUpdate() {
        this.updatedAt = LocalDateTime.now();
    }
}
```

Listing 4.2 – Extrait de Vehicle.java

4.3 Correspondance entité / table

Mapping JPA — PostgreSQL

Champ Java	Colonne SQL	Type SQL
`id` (UUID)	`id_vehicule`	`UUID`
`immatriculation`	`immatriculation`	`VARCHAR(20)`
`marque`	`marque`	`VARCHAR(100)`
`modele`	`modele`	`VARCHAR(100)`
`annee` (Short)	`annee`	`SMALLINT`
`statut` (Enum)	`statut`	`statut_vehicule`
`kilometrage`	`kilometrage`	`INTEGER`
`type`	`type`	`VARCHAR(50)`
`createdAt`	`created_at`	`TIMESTAMPTZ`
`updatedAt`	`updated_at`	`TIMESTAMPTZ`

Chapitre 5

API REST — CRUD

Le contrôleur REST expose les opérations CRUD classiques via `VehicleController`.

5.1 Endpoints disponibles

Endpoints REST du service véhicule

Méthode	URL	Action	Description
POST	`/api/vehicles`	Create	Créer un nouveau véhicule
GET	`/api/vehicles`	Read All	Lister tous les véhicules
GET	`/api/vehicles/{id}`	Read One	Obtenir un véhicule par UUID
GET	`/api/vehicles/statut/{s}`	Filter	Filtrer par statut
PUT	`/api/vehicles/{id}`	Update	Mettre à jour un véhicule
DELETE	`/api/vehicles/{id}`	Delete	Supprimer un véhicule

5.2 Contrôleur REST

```java
@RestController
@RequestMapping("/api/vehicles")
public class VehicleController {

    private final VehicleService vehicleService;

    @PostMapping
    public ResponseEntity<VehicleResponseDto> createVehicle(
            @Valid @RequestBody VehicleRequestDto request) {
        VehicleResponseDto response =
            vehicleService.createVehicle(request);
        return ResponseEntity.status(HttpStatus.CREATED)
                                .body(response);
```

```java
    }

    @GetMapping("/{id}")
    public ResponseEntity<VehicleResponseDto> getVehicleById(
            @PathVariable UUID id) {
        VehicleResponseDto vehicle =
            vehicleService.getVehicleById(id);
        return ResponseEntity.ok(vehicle);
    }

    @PutMapping("/{id}")
    public ResponseEntity<VehicleResponseDto> updateVehicle(
            @PathVariable UUID id,
            @Valid @RequestBody VehicleRequestDto request) {
        VehicleResponseDto response =
            vehicleService.updateVehicle(id, request);
        return ResponseEntity.ok(response);
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteVehicle(
            @PathVariable UUID id) {
        vehicleService.deleteVehicle(id);
        return ResponseEntity.noContent().build();
    }
}
```

Listing 5.1 – VehicleController.java

5.3 DTOs (Data Transfer Objects)

Le projet utilise deux DTOs pour découpler l'API de l'entité interne :

- **VehicleRequestDto** — Données entrantes avec validation :
  - `@NotBlank` sur marque, modèle, immatriculation, type
  - `@NotNull` sur l'année
  - `@Min(0)` sur le kilométrage
- **VehicleResponseDto** — Données sortantes avec ID UUID, timestamps, etc.

5.4 Couche Service

```java
@Service
@Transactional
public class VehicleServiceImpl implements VehicleService {

    private final VehicleRepository vehicleRepository;
    private final VehicleProducer vehicleProducer;

```

```java
    @Override
    public VehicleResponseDto createVehicle(
            VehicleRequestDto request) {
        // Verification d'unicite de l'immatriculation
        if (vehicleRepository.existsByImmatriculation(
                request.getImmatriculation())) {
            throw new IllegalArgumentException(
                "Immatriculation deja existante");
        }
        Vehicle vehicle = mapToEntity(request);
        Vehicle saved = vehicleRepository.save(vehicle);

        // Publication de l'evenement Kafka
        vehicleProducer.sendVehicleEvent(
            "VEHICLE_CREATED", mapToDto(saved));
        return mapToDto(saved);
    }
}
```

Listing 5.2 – Extrait de VehicleServiceImpl.java

Points importants de la couche Service :

- `@Transactional` assure l'intégrité des opérations
- `@Transactional(readOnly = true)` optimise les lectures
- Vérifications métier (unicité d'immatriculation)
- Publication automatique d'événements Kafka après chaque opération

5.5 Validation de l'API avec Bruno

Afin de valider le bon fonctionnement de notre API REST et sa connexion effective à la base de données PostgreSQL, nous avons effectué des requêtes manuelles à l'aide de l'outil de test d'API **Bruno**.

La figure 5.1 montre la création réussie d'un nouveau véhicule via une requête HTTP `POST`. Le serveur répond correctement avec un code `201 Created` et retourne le DTO complété avec l'UUID fraîchement généré en base de données, ainsi que les horodatages de création.

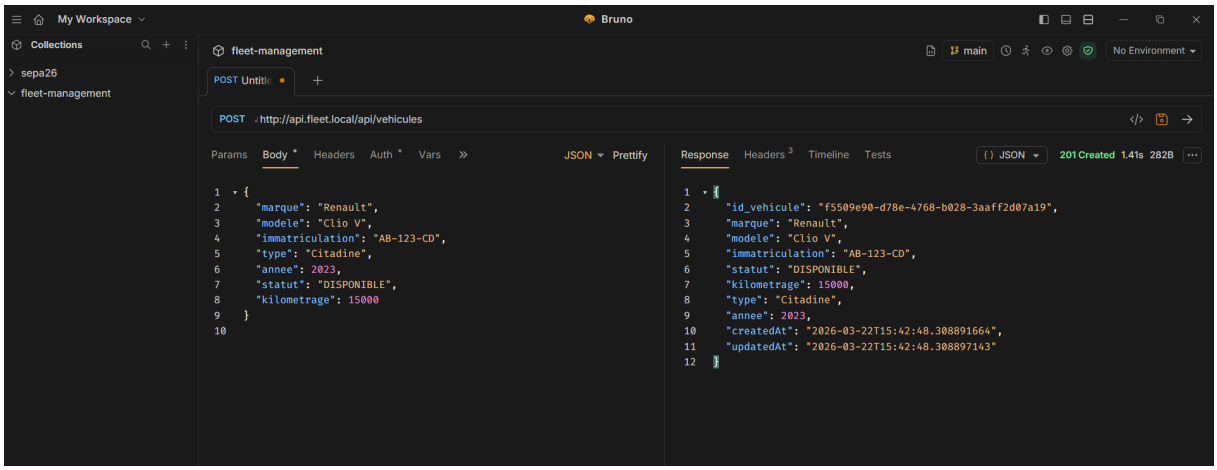


FIGURE 5.1 – Création d'un véhicule (POST) validée avec Bruno

La figure 5.2 illustre la récupération des véhicules via une requête HTTP `GET`. L'API répond avec un code `200 OK` et retourne le tableau JSON contenant le véhicule précédemment inséré, confirmant ainsi que la persistance des données et le mapping DTO fonctionnent parfaitement.

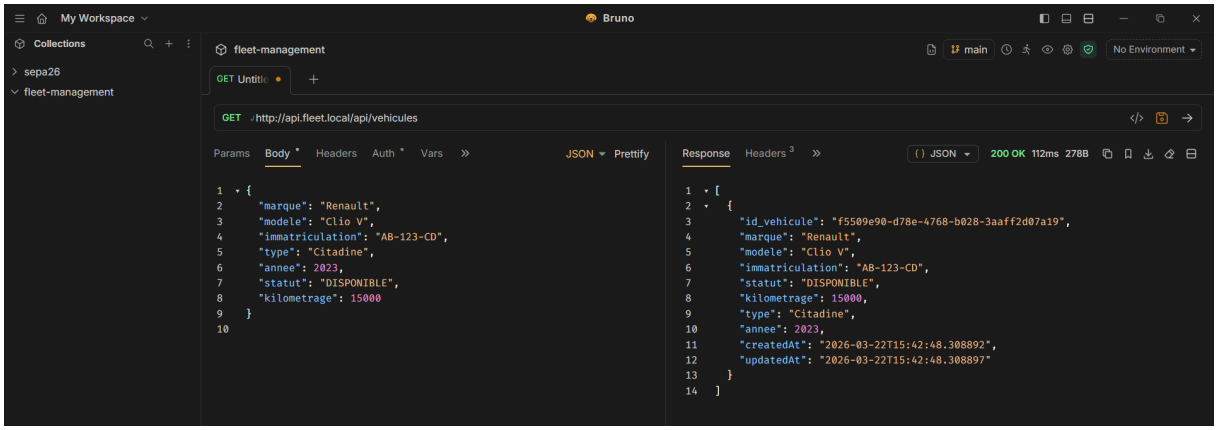


FIGURE 5.2 – Récupération de la liste des véhicules (GET) validée avec Bruno

Chapitre 6

Apache Kafka — Communication asynchrone

6.1 Rôle de Kafka dans l'architecture

Apache Kafka est utilisé comme système de messagerie asynchrone pour :

- **Découpler** les services — le producteur n'attend pas de réponse
- **Publier des événements** sur le cycle de vie des véhicules
- **Permettre le replay** des événements en cas de besoin
- **Alimenter d'autres services** (notifications, analytics, etc.)

6.2 Producteur Kafka

```java
@Service
public class VehicleProducer {

    private final KafkaTemplate<String, String> kafkaTemplate;
    private final ObjectMapper objectMapper;

    @Value("${app.kafka.topic.vehicle-events:vehicle-events}")
    private String vehicleEventsTopic;

    public void sendVehicleEvent(String eventType,
                                 VehicleResponseDto vehicle) {
        String key = eventType + "-" + vehicle.getId();
        String payload = objectMapper.writeValueAsString(
            new VehicleEvent(eventType, vehicle));
        kafkaTemplate.send(vehicleEventsTopic, key, payload);
    }

    // Classe interne -- structure de l'evenement
    public static class VehicleEvent {
        private String eventType;
        private VehicleResponseDto vehicle;
        private long timestamp;
```

```
    }
}
```

Listing 6.1 – VehicleProducer.java

Types d'événements Kafka

Événement	Déclencheur
VEHICLE_CREATED	Création d'un véhicule
VEHICLE_UPDATED	Mise à jour d'un véhicule
VEHICLE_DELETED	Suppression d'un véhicule

6.3 Consommateur Kafka

```java
@Service
public class VehicleConsumer {

    @KafkaListener(
        topics = "${app.kafka.topic.vehicle-events}",
        groupId = "${spring.kafka.consumer.group-id}")
    public void consumeVehicleEvent(String message) {
        log.info("Evenement vehicule recu: {}", message);
        // Traitement : notifications, mises a jour, etc.
    }
}
```

Listing 6.2 – VehicleConsumer.java

6.4 Configuration Kafka

```yaml
spring:
  kafka:
    bootstrap-servers: localhost:9092
    producer:
      key-serializer: org.apache.kafka.common.serialization
                      .StringSerializer
      value-serializer: org.apache.kafka.common.serialization
                        .StringSerializer
    consumer:
      group-id: vehicule-service-group
      auto-offset-reset: earliest
```

Listing 6.3 – Configuration Kafka dans application.yml

Chapitre 7

GraphQL — API flexible

7.1 Pourquoi GraphQL ?

GraphQL offre des avantages complémentaires à REST :

— **Requêtes flexibles** — Le client choisit exactement les champs voulus

— **Un seul endpoint** — `/graphql` remplace plusieurs endpoints REST

— **Pas de sur-récupération** — Contrairement à REST qui retourne tous les champs

— **Introspection** — Documentation automatique du schéma

7.2 Schéma GraphQL

```
type Vehicle {
    id: ID!
    marque: String!
    modele: String!
    immatriculation: String!
    statut: String!
    kilometrage: Int
    type: String!
    annee: Int!
    createdAt: String
    updatedAt: String
}

type Query {
    allVehicles: [Vehicle!]!
    vehicleById(id: ID!): Vehicle
    vehiclesByStatut(statut: String!): [Vehicle!]!
}

type Mutation {
    createVehicle(
        marque: String!, modele: String!,
        immatriculation: String!, type: String!,
        statut: String, kilometrage: Int, annee: Int!
```

```
    ): Vehicle!
    updateVehicle(
        id: ID!, marque: String!, modele: String!,
        immatriculation: String!, type: String!,
        statut: String, kilometrage: Int, annee: Int!
    ): Vehicle!
    deleteVehicle(id: ID!): Boolean!
}
```

Listing 7.1 – schema.graphqls

7.3 Contrôleur GraphQL

```java
@Controller
public class VehicleGraphQLController {

    private final VehicleService vehicleService;

    @QueryMapping
    public List<VehicleResponseDto> allVehicles() {
        return vehicleService.getAllVehicles();
    }

    @QueryMapping
    public VehicleResponseDto vehicleById(@Argument String id) {
        return vehicleService.getVehicleById(
            UUID.fromString(id));
    }

    @MutationMapping
    public VehicleResponseDto createVehicle(
            @Argument String marque,
            @Argument String modele,
            @Argument String immatriculation,
            @Argument String type,
            @Argument Integer annee) {
        VehicleRequestDto request = new VehicleRequestDto();
        request.setMarque(marque);
        request.setModele(modele);
        request.setImmatriculation(immatriculation);
        request.setType(type);
        request.setAnnee(annee != null
            ? annee.shortValue() : null);
        return vehicleService.createVehicle(request);
    }
}
```

Listing 7.2 – Extrait de VehicleGraphQLController.java

7.4 Exemple de requête GraphQL

```
query {
  allVehicles {
    marque
    modele
    statut
  }
}
```

Listing 7.3 – Requête pour obtenir marque et modèle

L'interface **GraphiQL** est activée et accessible à `http://localhost:4000/graphiql` pour tester les requêtes de manière interactive.

7.5 Test des Resolvers avec Apollo Sandbox et Keycloak

Afin de valider le bon fonctionnement de notre API GraphQL et vérifier que la couche de sécurité Keycloak est correctement appliquée sur nos points de terminaison, nous utilisons **Apollo Sandbox**. Cet outil permet d'exécuter des requêtes et des mutations interactives tout en gérant les en-têtes HTTP nécessaires à l'authentification.

7.5.1 Configuration de l'environnement de test

Puisque le service de gestion de flotte exige une authentification pour modifier les données, il est indispensable de fournir un jeton d'accès valide lors des tests interactifs.

1. **Générer un token Keycloak :** Récupérer un `access_token` valide en utilisant l'un des comptes de test autorisés (par exemple, le rôle `manager` ou `admin`) via une requête POST.
2. **Configurer l'en-tête dans Apollo :** Dans l'interface d'Apollo Sandbox (connectée à l'URL `http://localhost:4000/`), ouvrir l'onglet **Headers**.
3. Ajouter une nouvelle variable d'en-tête avec la clé `Authorization` et pour valeur `Bearer <VOTRE_TOKEN_JWT>`.

7.5.2 Validation de la Mutation `createVehicule`

Une fois l'authentification configurée, nous pouvons tester l'exécution de nos resolvers. L'exemple ci-dessous illustre la création réussie d'un véhicule, prouvant que le resolver GraphQL communique correctement avec la couche de service tout en franchissant la barrière de sécurité.

```
mutation CreateVehicule {
  createVehicule(input: {
```

```
    immatriculation: "XY-987-ZZ",
    marque: "Peugeot",
    modele: "3008",
    annee: 2023,
    type: "SUV",
    kilometrage: 1500
  }) {
    id_vehicule
    immatriculation
    marque
    statut
  }
}
```

Listing 7.4 – Requête GraphQL (Mutation) pour créer un véhicule

Le serveur valide la signature du jeton, exécute la mutation via le resolver correspondant, et retourne un statut HTTP 200 avec le résultat suivant :

```
{
  "data": {
    "createVehicule": {
      "id_vehicule": "21355dd1-3374-4a5a-9532-842d2e497915",
      "immatriculation": "XY-987-ZZ",
      "marque": "Peugeot",
      "statut": "DISPONIBLE"
    }
  }
}
```

Listing 7.5 – Résultat de l'exécution de la mutation

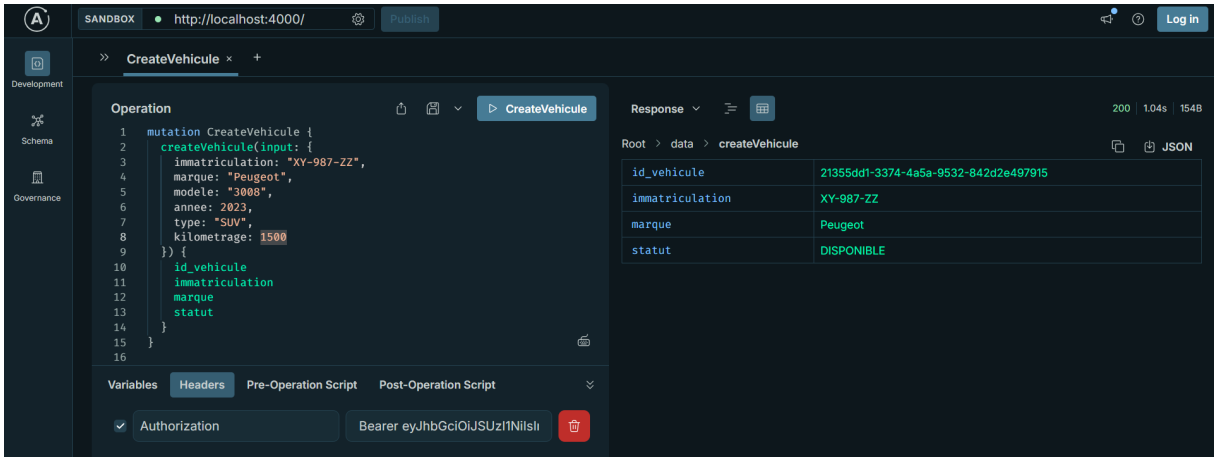


FIGURE 7.1 – Test interactif de création avec authentification via Apollo Sandbox

Ce test interactif confirme le routage GraphQL correct, la validation réussie du token JWT par Spring Security, et la persistance des données en base PostgreSQL.

Chapitre 8

Tests automatisés

8.1 Stratégie de test

Le projet implémente deux niveaux de tests :

1. **Tests unitaires** — Couche service avec Mockito
2. **Tests d'intégration** — Couche contrôleur avec MockMvc

8.2 Tests unitaires de la couche Service

Les tests de `VehicleServiceTest` utilisent **Mockito** pour isoler la logique métier :

```
@ExtendWith(MockitoExtension.class)
class VehicleServiceTest {

    @Mock
    private VehicleRepository vehicleRepository;
    @Mock
    private VehicleProducer vehicleProducer;
    @InjectMocks
    private VehicleServiceImpl vehicleService;

    @Test
    void createVehicle_Success() {
        when(vehicleRepository.existsByImmatriculation(
            "AB-123-CD")).thenReturn(false);
        when(vehicleRepository.save(any(Vehicle.class)))
            .thenReturn(vehicle);

        VehicleResponseDto result =
            vehicleService.createVehicle(requestDto);

        assertNotNull(result);
        assertEquals("Renault", result.getMarque());
        verify(vehicleProducer).sendVehicleEvent(
            eq("VEHICLE_CREATED"),
            any(VehicleResponseDto.class));
```

```
    }

    @Test
    void createVehicle_DuplicateImmatriculation() {
        when(vehicleRepository.existsByImmatriculation(
            "AB-123-CD")).thenReturn(true);

        assertThrows(IllegalArgumentException.class,
            () -> vehicleService.createVehicle(requestDto));
        verify(vehicleRepository, never())
            .save(any(Vehicle.class));
    }
}
```

Listing 8.1 – Extrait de VehicleServiceTest.java

8.3 Tests d'intégration du contrôleur REST

```
@WebMvcTest(VehicleController.class)
class VehicleControllerTest {

    @Autowired
    private MockMvc mockMvc;
    @MockBean
    private VehicleService vehicleService;

    @Test
    void createVehicle_ReturnsCreated() throws Exception {
        when(vehicleService.createVehicle(
            any(VehicleRequestDto.class)))
            .thenReturn(responseDto);

        mockMvc.perform(post("/api/vehicles")
                .contentType(MediaType.APPLICATION_JSON)
                .content(objectMapper
                    .writeValueAsString(requestDto)))
            .andExpect(status().isCreated())
            .andExpect(jsonPath("$.marque").value("Renault"))
            .andExpect(jsonPath("$.immatriculation")
                .value("AB-123-CD"));
    }

    @Test
    void createVehicle_ValidationError() throws Exception {
        VehicleRequestDto invalid = new VehicleRequestDto();

        mockMvc.perform(post("/api/vehicles")
                .contentType(MediaType.APPLICATION_JSON)
                .content(objectMapper
```

```
                        .writeValueAsString(invalid)))
                .andExpect(status().isBadRequest());
    }
}
```

Listing 8.2 – Extrait de VehicleControllerTest.java

8.4 Cas de test couverts

Couverture des tests

Classe	Test	Scénario
Service	createVehicle_Success	Création réussie + événement Kafka
Service	createVehicle_Duplicate	Rejet si immatriculation dupliquée
Service	getVehicleById_Success	Récupération par UUID
Service	getVehicleById_NotFound	Exception si inexistant
Service	getAllVehicles_Success	Liste complète
Service	getVehiclesByStatut	Filtrage par statut
Service	updateVehicle_Success	Mise à jour + événement Kafka
Service	updateVehicle_NotFound	Exception si inexistant
Service	deleteVehicle_Success	Suppression + événement Kafka
Service	deleteVehicle_NotFound	Exception si inexistant
Controller	createVehicle_Created	HTTP 201 + JSON
Controller	getAllVehicles_List	HTTP 200 + tableau
Controller	getVehicleById	HTTP 200 + détail
Controller	updateVehicle_Updated	HTTP 200 + modifié
Controller	deleteVehicle_NoContent	HTTP 204
Controller	validation_BadRequest	HTTP 400 si invalide

8.5 Mise en œuvre et Tests du Service Véhicule

Dans le cadre de la semaine 3, le `vehicule-service` a été mis à jour pour intégrer les événements Kafka, les résolveurs GraphQL et la télémétrie OpenTelemetry. L'objectif de couverture de code de 80% a été atteint.

8.5.1 Corrections et Améliorations

Plusieurs ajustements ont été nécessaires pour stabiliser la suite de tests et assurer la conformité avec les APIs récentes :

— **GraphQL :** Mise à jour des tests pour utiliser `path().entity()` suite à la dépréciation de `node()` dans `GraphQlTester` (Spring GraphQL 3.1+).

— **Kafka :** Résolution d'un problème d'injection de configuration dans `VehicleProducerTest` via `ReflectionTestUtils` pour simuler correctement les topics.

— **Gestion des Erreurs :** Correction des types génériques dans `GlobalExceptionHandlerTest` pour une meilleure robustesse.

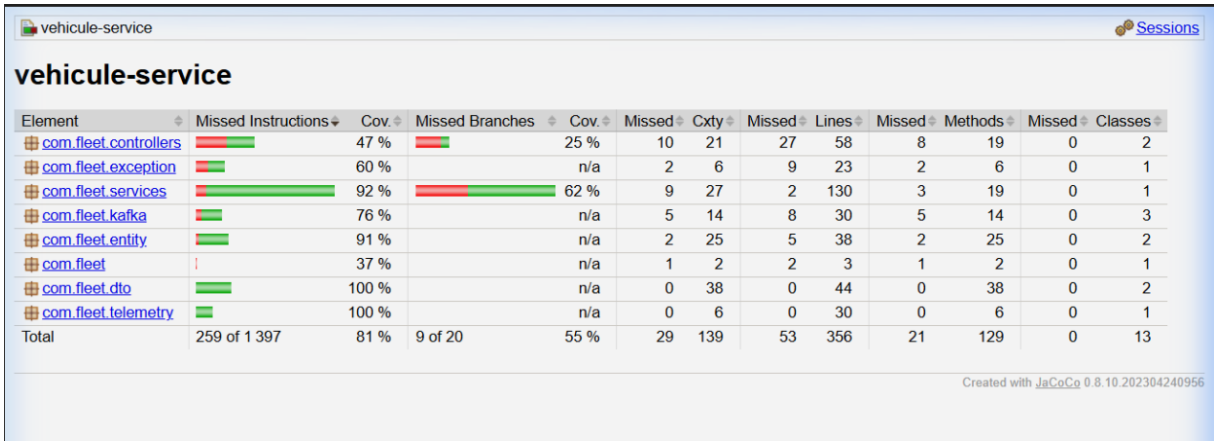
— **Stabilité :** Configuration du profil `test` avec une base de données H2 en mémoire pour garantir l'isolation des tests unitaires.

8.5.2 Résultats des Tests et Couverture

La suite de tests complète (23 tests) s'exécute avec succès. Les métriques de couverture générées par JaCoCo sont les suivantes :

Métrique	Valeur atteinte
Couverture des Instructions (Instructions)	**81%**
Couverture des Lignes (Lines)	**85%**
Couverture des Méthodes (Methods)	83.7%
Couverture des Classes (Classes)	100%

TABLE 8.1 – Résumé de la couverture de code du vehicule-service



vehicule-service Sessions

vehicule-service

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.fleet.controllers		47 %		25 %	10	21	27	58	8	19	0	2
com.fleet.exception		60 %		n/a	2	6	9	23	2	6	0	1
com.fleet.services		92 %		62 %	9	27	2	130	3	19	0	1
com.fleet.kafka		76 %		n/a	5	14	8	30	5	14	0	3
com.fleet.entity		91 %		n/a	2	25	5	38	2	25	0	2
com.fleet		37 %		n/a	1	2	2	3	1	2	0	1
com.fleet.dto		100 %		n/a	0	38	0	44	0	38	0	2
com.fleet.telemetry		100 %		n/a	0	6	0	30	0	6	0	1
Total	259 of 1 397	81 %	9 of 20	55 %	29	139	53	356	21	129	0	13

Created with JaCoCo 0.8.10.202304240956

FIGURE 8.1 – Capture d'écran du rapport JaCoCo montrant 81% de couverture.

Listing 8.3 – Sortie de l'exécution des tests Maven

```
[INFO] Results:
[INFO] Tests run: 23, Failures: 0, Errors: 0, Skipped: 0
[INFO] ------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------
```

8.5.3 Détails des Tests Effectués

La suite de tests a été conçue pour couvrir l'ensemble de la pyramide des tests, des services unitaires aux contrôleurs d'API.

1. **VehicleServiceTest (Tests de Logique Métier) :**

- Vérifie la création d'un véhicule avec validation des données métier.
- Teste la levée d'exceptions en cas de véhicule non trouvé ou d'immatriculation en doublon.
- Valide que les appels aux services tiers (Kafka, Télémétrie) sont correctement effectués lors des actions de modification.

2. **VehicleControllerTest (Tests API REST) :**
   - Vérifie les points de terminaison standard (CRUD) : `GET`, `POST`, `DELETE`.
   - Teste que les codes de statut HTTP (`200 OK`, `201 Created`, `404 Not Found`) sont corrects selon les entrées fournies.
3. **VehicleGraphQLControllerTest (Tests GraphQL) :**
   - Teste les requêtes (`queries`) pour récupérer un véhicule par ID ou la liste complète.
   - Teste les `mutations` pour la création d'un véhicule via le schéma GraphQL.
   - Valide le mappage correct des champs entre l'entité et le schéma GraphQL.
4. **VehicleProducerTest (Tests de Messagerie Kafka) :**
   - Simule l'envoi d'événements (`VEHICLE_CREATED`, `VEHICLE_UPDATED`).
   - Vérifie que le format du message envoyé (JSON) contient bien les informations requises.
   - S'assure que le message est envoyé sur le bon topic Kafka.
5. **VehicleTelemetryTest (Tests d'Observabilité) :**
   - Vérifie que les *spans* de télémétrie sont correctement créés lors des opérations critiques.
   - Teste l'enregistrement des attributs personnalisés (ex : ID du véhicule, type d'opération) dans OpenTelemetry.
6. **GlobalExceptionHandlerTest (Tests de Robustesse) :**
   - Vérifie que toutes les exceptions non gérées sont interceptées.
   - S'assure que le service renvoie une structure d'erreur cohérente au client (`timestamp`, `message`, `code d'erreur`).

Chapitre 9

Observabilité — OpenTelemetry SDK

L'implémentation de l'observabilité repose sur les **3 piliers d'OpenTelemetry** (Traces, Métriques, Logs), intégrés via le SDK Java.

9.1 Le SDK et la configuration centralisée

Une classe `VehicleTelemetry` centralise l'accès au **Tracer** (pour les traces) et au **Meter** (pour les métriques).

```
@Component
public class VehicleTelemetry {
    private final Tracer tracer;
    private final LongCounter vehiclesCreatedCounter;
    private final LongHistogram operationDurationHistogram;

    public VehicleTelemetry(OpenTelemetry openTelemetry) {
        this.tracer = openTelemetry.getTracer("vehicule-service");
        Meter meter = openTelemetry.getMeter("vehicule-service");

        // Compteur : nombre total de creations
        this.vehiclesCreatedCounter = meter.counterBuilder("
           vehicles.created.total").build();

        // Histogramme : duree des operations
        this.operationDurationHistogram = meter.histogramBuilder("
           vehicles.operation.duration").ofLongs().build();
    }
}
```

Listing 9.1 – Extraits de VehicleTelemetry.java

9.2 Pilier 1 : Traces Distribuées

Les traces permettent de suivre une requête. Nous utilisons deux méthodes :

— **Instrumentation déclarative** : Utilisation de `@WithSpan` pour créer automatiquement un span sur une méthode.

— **Attributs personnalisés** : Utilisation de `@SpanAttribute` pour injecter des paramètres (ex : UUID du véhicule) dans la trace.

```
@WithSpan("createVehicle")
public VehicleResponseDto createVehicle(VehicleRequestDto request)
    {
     Span.current().setAttribute("vehicle.marque", request.getMarque
        ());
     // ... logique metier ...
}
```

Listing 9.2 – Traces dans la couche Service

9.3 Pilier 2 : Métriques Personnalisées

Contrairement aux traces, les métriques sont des données numériques agrégées. Nous mesurons :

— Le nombre total de véhicules créés et supprimés.

— Le taux d'erreurs par type d'opération.

— La distribution des temps de traitement (latence).

9.4 Pilier 3 : Logs Structurés et Corrélation

Pour relier les logs aux traces, nous injectons le `traceId` et le `spanId` dans le contexte de diagnostic (**MDC**).

```
<encoder>
    <pattern>
        %d %-5level [traceId=%X{traceId:-none}, spanId=%X{spanId:-
            none}] %logger - %msg%n
    </pattern>
</encoder>
```

Listing 9.3 – Configuration Logback (logback-spring.xml)

9.5 Vérification du fonctionnement

Pour s'assurer que l'observabilité est active :

Plan de vérification

1. **Logs** : Lancer l'application et effectuer un appel API. Le log doit afficher `traceId=...` avec un ID hexadécimal au lieu de `none`.
2. **Métriques** : Accéder à `/actuator/prometheus`. Rechercher la ligne `vehicles_created_total` pour voir le compteur augmenter.
3. **Traces** : Si un collecteur (Jaeger/Tempo) est configuré, les spans `createVehicle` apparaîtront avec leurs attributs (`vehicle.marque`).

Chapitre 10

Configuration de l'application

```yaml
server:
  port: 4000

spring:
  application:
    name: vehicule-service
  datasource:
    url: jdbc:postgresql://localhost:5432/fleet_vehicles
    username: postgres
    password: postgres
  jpa:
    hibernate:
      ddl-auto: update
    properties:
      hibernate:
        dialect: org.hibernate.dialect.PostgreSQLDialect
  graphql:
    graphiql:
      enabled: true
    path: /graphql

management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus
```

Listing 10.1 – application.yml – Configuration complète

Note : Le mode `ddl-auto: update` permet à Hibernate de mettre à jour le schéma automatiquement au démarrage. En production, il est recommandé d'utiliser un outil de migration comme **Flyway** ou **Liquibase**.

Chapitre 11

Authentification et Sécurité — Keycloak

11.1 Configuration de base et Rôles

Le système de gestion de flotte utilise Keycloak pour centraliser l'authentification et l'autorisation de tous les microservices.

Paramètres Keycloak

Paramètre	Valeur
URL locale (hôte)	`http://localhost:9080`
URL interne Docker	`http://keycloak:8080`
Realm	`fleet-management`
Admin console	`http://localhost:9080/admin`

Règle importante sur les URLs et Docker : Pour éviter les erreurs `Connection refused` ou les problèmes d'**Issuer Mismatch**, utilisez `http://localhost:9080` pour l'**Issuer URI** et `http://host.docker.internal:9080` pour l'URL de téléchargement des clés **JWKS**.

Le realm `fleet-management` contient 4 rôles métier : `admin`, `manager`, `technicien`, et `utilisateur`. Des utilisateurs de test (`admin-fleet`, `manager-fleet`, etc.) sont configurés pour simuler ces rôles.

11.2 Intégration Spring Boot (Java)

11.2.1 Dépendances et Configuration YAML

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
```

```
    <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
</dependency>
```

Listing 11.1 – Extrait pom.xml pour Spring Security

```
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: http://localhost:9080/realms/fleet-management
          jwk-set-uri: http://host.docker.internal:9080/realms/fleet-management/protocol/openid-connect/certs
```

Listing 11.2 – Configuration de la sécurité dans application.yml

11.2.2 Extraction des Rôles et Filtres de Sécurité

Le convertisseur suivant permet d'extraire les rôles du jeton Keycloak et de les adapter pour `@PreAuthorize` dans Spring Security.

```
@Component
public class KeycloakRoleConverter implements Converter<Jwt, Collection<GrantedAuthority>> {

    @Override
    public Collection<GrantedAuthority> convert(Jwt jwt) {
        Map<String, Object> realmAccess = jwt.getClaim("realm_access");
        if (realmAccess == null || !realmAccess.containsKey("roles")) {
            return List.of(new SimpleGrantedAuthority("ROLE_admin"));
        }
        List<String> roles = (List<String>) realmAccess.get("roles");
        return roles.stream()
                .map(role -> new SimpleGrantedAuthority("ROLE_" + role))
                .collect(Collectors.toList());
    }
}
```

Listing 11.3 – KeycloakRoleConverter.java

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class SecurityConfig {
```

```
5
6     @Autowired
7     private KeycloakRoleConverter keycloakRoleConverter;
8
9     @Bean
10    public SecurityFilterChain filterChain(HttpSecurity http)
         throws Exception {
11        http
12            .csrf(csrf -> csrf.disable())
13            .sessionManagement(session ->
14                session.sessionCreationPolicy(SessionCreationPolicy
                     .STATELESS))
15            .authorizeHttpRequests(auth -> auth
16                .requestMatchers("/actuator/health").permitAll()
17                .anyRequest().authenticated()
18            )
19            .oauth2ResourceServer(oauth2 -> oauth2
20                .jwt(jwt -> jwt.jwtAuthenticationConverter(
                     jwtAuthenticationConverter()))
21            );
22        return http.build();
23    }
24
25    @Bean
26    public JwtAuthenticationConverter jwtAuthenticationConverter()
         {
27        JwtAuthenticationConverter converter = new
             JwtAuthenticationConverter();
28        converter.setJwtGrantedAuthoritiesConverter(
             keycloakRoleConverter);
29        return converter;
30    }
31 }
```

Listing 11.4 – SecurityConfig.java

11.3 Intégration dans d'autres environnements

Le serveur Keycloak est central, il protège également nos autres services développés dans des langages différents.

11.3.1 Node.js

Pour Node.js, on utilise les librairies `jsonwebtoken` et `jwks-rsa` pour valider manuellement la signature du jeton via un middleware :

Listing 11.5 – Installation des dépendances Node.js

```
1 npm install jwks-rsa jsonwebtoken
```

11.3.2 Python (FastAPI)

Pour les services en Python, on utilise `python-jose` et `fastapi.security.HTTPBearer` pour injecter l'utilisateur décodé directement dans les routes.

Listing 11.6 – Installation des dépendances Python

```
pip install python-jose[cryptography] httpx fastapi
```

11.4 Commandes utiles et Dépannage

Listing 11.7 – Obtenir un token via le terminal

```
TOKEN=$(curl -s -X POST \
  'http://localhost:9080/realms/fleet-management/protocol/openid-
     connect/token' \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  -d 'client_id=admin-cli&username=manager-fleet&password=
     Manager1234!&grant_type=password' \
  | python3 -c "import sys,json; print(json.load(sys.stdin)['
     access_token'])")

echo $TOKEN
```

Problèmes courants Keycloak

Erreur rencontrée	Solution probable
`401 Unauthorized`	Token absent ou expiré (durée de vie 15 min).
`403 Forbidden`	L'utilisateur n'a pas le rôle requis dans `@PreAuthorize`.
`Issuer mismatch`	Bien différencier `localhost` (issuer) et `host.docker.internal` (jwks-uri) dans le YAML.

Chapitre 12

Conclusion

Le microservice **vehicule-service** implémente une architecture moderne et complète :

- **CRUD REST** structuré avec validation, DTOs et gestion d'erreurs
- **GraphQL** offrant une API flexible complémentaire
- **Kafka** pour la communication événementielle asynchrone inter-services
- **Tests automatisés** couvrant la logique métier et les endpoints HTTP
- **Observabilité** via OpenTelemetry, Actuator et Prometheus
- **Sécurité** gérée par Keycloak et Spring Security (OAuth2 / JWT)
- **Cohérence entité/base** avec un mapping JPA aligné sur le schéma PostgreSQL

Cette architecture permet au service d'être *scalable*, *maintenable* et *observable* dans un contexte de déploiement microservices.